

IMPLEMENTASI ALGORITMA K-NEAREST NEIGHBORS (KNN) DALAM KLASIFIKASI PERFORMA AKADEMIK SISWA PADA DATASET STUDENT PERFORMANCE

Andre Astamam¹⁾, Baiq Erwina Yolanda²⁾, Fauzan Hari Ramdani³⁾, Lalu Galang Abdullah.⁴⁾

1,2,3,4)Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: andreastamam14@gmail.com, baiqerwinayolanda26@gmail.com,
fauzanhari4@gmail.com, abdullahgalang06@gmail.com

Abstrak

Performa akademik siswa merupakan salah satu indikator penting dalam menganalisis perkembangan dan potensi siswa, namun pengolahan data berskala besar secara sekuensial sering menyebabkan waktu eksekusi yang tidak efisien. Penelitian ini menerapkan algoritma K-Nearest Neighbors (KNN) yang dipadukan dengan komputasi paralel menggunakan Message Passing Interface (MPI) untuk meningkatkan efisiensi klasifikasi. Dataset yang digunakan terdiri dari 300.000 data latih dan 75.000 data uji, diproses secara terdistribusi pada beberapa core melalui mekanisme pembagian partisi MPI. Hasil percobaan menunjukkan bahwa variasi jumlah core (N) mencapai waktu eksekusi terbaik pada N=2 sebesar 262,00 detik, namun pada N selanjutnya waktu eksekusi kembali meningkat hingga 316,43 detik pada N=7, yang mengindikasikan adanya overhead komunikasi antar-proses seiring bertambahnya jumlah core. Pada variasi nilai K, akurasi meningkat dari 96,11% pada K=1 hingga 96,82% pada K=9, menandakan kestabilan prediksi yang lebih baik pada nilai K yang lebih besar. Variasi jumlah data latih memperlihatkan peningkatan waktu eksekusi secara konsisten dari 93,64 detik pada 150.000 data hingga 265,02 detik pada 300.000 data, konsisten pada seluruh konfigurasi core yang diuji. Penelitian ini membuktikan bahwa kombinasi KNN dan MPI efektif dalam menangani data berskala besar dengan performa klasifikasi yang stabil.

Kata kunci: K-Nearest Neighbors; Klasifikasi; Komputasi Paralel; Message Passing Interface; Waktu Eksekusi.

Abstract

Student academic performance is one of the key indicators in analyzing student development and potential; however, sequential processing of large-scale data often leads to inefficient execution times. This study applies the K-Nearest Neighbors (KNN) algorithm combined with parallel computing using the Message Passing Interface (MPI) to improve classification efficiency. The dataset consists of 300,000 training samples and 75,000 test samples, processed in a distributed manner across multiple cores through MPI partitioning mechanisms. Experimental results show that varying the number of cores (N) achieves the best execution time at N=2 with 262.00 seconds, while further increasing N raises execution time back up to 316.43 seconds at N=7, indicating growing inter-process communication overhead as the number of cores increases. In the variation of K values, accuracy improves from 96.11% at K=1 to 96.82% at K=9, reflecting greater prediction stability at higher K values. Variation in training data size demonstrates a proportional increase in execution time from 93.64 seconds at 150,000 samples to 265.02 seconds at 300,000 samples, consistent across all tested core configurations. This study demonstrates that the combination of KNN and MPI is effective in handling large-scale data with stable classification performance.

Keyword: Classification; Execution Time; K-Nearest Neighbors; Message Passing Interface; Parallel Computing.

I. PENDAHULUAN

Perkembangan teknologi informasi telah mendorong institusi pendidikan untuk menghasilkan data siswa dalam jumlah yang semakin besar. Data tersebut umumnya mencakup nilai akademik, jumlah kehadiran, serta aktivitas belajar siswa [1]. Meskipun data tersebut berpotensi memberikan wawasan mendalam mengenai perkembangan siswa, dalam praktiknya data tersebut sering kali belum dimanfaatkan secara optimal untuk keperluan analisis lanjutan [2].

Proses analisis secara manual menjadi kurang efektif seiring meningkatnya volume data. Banyaknya atribut dalam data siswa turut membuat evaluasi menjadi lebih kompleks, sehingga menyulitkan identifikasi pola yang dapat digunakan sebagai dasar pengambilan keputusan [3]. Oleh karena itu, diperlukan pendekatan yang mampu mengolah data secara otomatis dan sistematis, salah satunya adalah teknik klasifikasi dalam data mining [1].

Algoritma K-Nearest Neighbors (KNN) merupakan salah satu metode klasifikasi yang sederhana namun efektif [3]. Beberapa penelitian menunjukkan bahwa KNN mampu memberikan hasil klasifikasi yang baik pada data pendidikan dengan tingkat akurasi yang cukup tinggi [4]. Namun, seiring meningkatnya jumlah data, proses perhitungan jarak pada KNN dapat menjadi lebih lambat [5]. Untuk mengatasi kelemahan tersebut, diperlukan pendekatan komputasi paralel menggunakan Message Passing Interface (MPI), yang memungkinkan pembagian beban komputasi ke beberapa proses sehingga dapat mempercepat waktu eksekusi [6].

Penelitian ini bertujuan untuk mengimplementasikan algoritma KNN dalam mengklasifikasikan performa akademik siswa menggunakan dataset Student Performance, menganalisis tingkat akurasi yang dihasilkan, mengimplementasikan komputasi paralel dengan MPI pada algoritma KNN, dan membandingkan efisiensi waktu eksekusi antara pendekatan sekuensial dan paralel.

Penelitian ini diharapkan dapat memberikan kontribusi terhadap pengembangan ilmu komputasi paralel, khususnya dalam penerapan algoritma K-Nearest Neighbors (KNN) berbasis MPI untuk

klasifikasi data pendidikan, serta menjadi referensi bagi penelitian selanjutnya yang berkaitan dengan optimasi performa algoritma machine learning.

II. TINJAUAN PUSTAKA

A. Data Mining dalam Bidang Pendidikan

Data mining merupakan proses penggalian informasi dari sekumpulan data berukuran besar untuk menemukan pola atau hubungan yang sebelumnya tidak diketahui [7]. Proses ini memanfaatkan berbagai teknik dari bidang statistik, machine learning, dan basis data untuk menghasilkan informasi yang berguna dalam pengambilan keputusan [7]. Penerapan data mining dalam bidang pendidikan telah banyak dilakukan oleh para peneliti. Setiyadi dan Nurdin (2012) menunjukkan bahwa data akademik siswa yang mencakup nilai, kehadiran, dan aktivitas belajar memiliki potensi besar untuk dianalisis lebih lanjut, namun dalam praktiknya sering kali belum dimanfaatkan secara optimal [2]. Suliman (2021) juga menerapkan data mining untuk menganalisis prestasi belajar mahasiswa berdasarkan faktor pergaulan dan sosial ekonomi, dan menyimpulkan bahwa pendekatan otomatis berbasis algoritma jauh lebih efektif dibandingkan evaluasi manual ketika volume data terus meningkat [8]. Hal ini menegaskan bahwa kebutuhan akan sistem klasifikasi yang efisien dan akurat dalam bidang pendidikan semakin mendesak seiring bertambahnya skala data.

B. Klasifikasi Data Akademik

Klasifikasi merupakan salah satu teknik dalam data mining yang digunakan untuk mengelompokkan data ke dalam kategori tertentu berdasarkan karakteristik yang dimilikinya [9]. Proses klasifikasi dilakukan dengan menggunakan data latih yang telah memiliki label, kemudian model yang dihasilkan digunakan untuk memprediksi kelas dari data baru [10]. Dalam konteks pendidikan, beberapa penelitian telah menerapkan teknik klasifikasi untuk memprediksi performa dan kelulusan siswa. Sari et al. (2023) menerapkan KNN untuk memprediksi kelulusan siswa Sekolah

Menengah Pertama dan memperoleh hasil prediksi yang cukup akurat [9]. Wardana dan Suherman (2025) turut membuktikan bahwa teknik klasifikasi berbasis data mining mampu menghasilkan pengelompokan yang sistematis dan dapat diandalkan untuk pengambilan keputusan [10]. Namun, penelitian-penelitian tersebut umumnya belum mempertimbangkan efisiensi waktu komputasi ketika jumlah data ditingkatkan ke skala yang lebih besar, yang menjadi salah satu fokus utama penelitian ini. Dalam penelitian ini, klasifikasi digunakan untuk mengelompokkan siswa ke dalam kategori nilai A, B, C, D, dan E berdasarkan data akademik yang tersedia.

C. Algoritma K-Nearest Neighbor (KNN)

K-Nearest Neighbor (KNN) merupakan algoritma klasifikasi yang bekerja berdasarkan konsep kedekatan jarak antar data [11]. Algoritma ini mengklasifikasikan data baru dengan melihat sejumlah K data terdekat (neighbor), kemudian menentukan kelas berdasarkan mayoritas dari tetangga terdekat tersebut. KNN termasuk dalam metode non-parametrik dan sering disebut sebagai lazy learning karena tidak melakukan proses pelatihan model secara eksplisit [12]. KNN telah banyak diterapkan dalam penelitian klasifikasi data pendidikan. Tatimu et al. (2020) mengimplementasikan KNN untuk menganalisis klasifikasi kelulusan mahasiswa di Fakultas Teknik Universitas Negeri Manado dan menunjukkan bahwa KNN efektif digunakan pada data akademik dengan atribut numerik [1]. Arrohman et al. (2023) menerapkan KNN dan Random Forest pada dataset Student Performance dan menemukan bahwa KNN mampu menghasilkan akurasi yang kompetitif pada data pendidikan berskala sedang [3]. Hidayat et al. (2023) juga mengonfirmasi bahwa performa KNN sangat dipengaruhi oleh pemilihan nilai K dan skala data yang digunakan [4].

Proses perhitungan KNN dilakukan saat tahap prediksi dengan menghitung jarak antara data uji dan seluruh data latih menggunakan Euclidean Distance [12], dengan rumus sebagai berikut:

$$d(x,y) = \sqrt{\sum_i (x_i - y_i)^2} \quad (1)$$

Meskipun demikian, kelemahan utama KNN adalah waktu komputasi yang meningkat seiring bertambahnya jumlah data, karena setiap data uji harus dibandingkan dengan seluruh data latih [12]. Hilmi et al. (2024) secara eksplisit mengidentifikasi bahwa waktu komputasi KNN meningkat signifikan ketika volume data latih diperbesar, sehingga diperlukan strategi optimasi untuk mengatasi keterbatasan ini pada dataset berskala besar [5]. Kelemahan inilah yang menjadi motivasi utama penelitian ini untuk mengintegrasikan KNN dengan pendekatan komputasi paralel.

D. Komputasi Paralel dan MPI

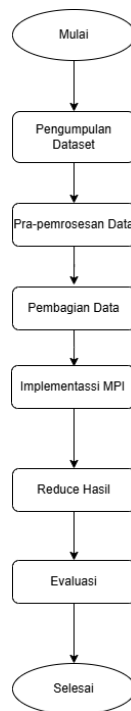
Komputasi paralel merupakan teknik pemrosesan data dengan cara membagi suatu pekerjaan menjadi beberapa bagian yang dapat dijalankan secara bersamaan [13]. Pendekatan ini digunakan untuk meningkatkan efisiensi dan mempercepat waktu eksekusi program, terutama pada pengolahan data dalam jumlah besar. Message Passing Interface (MPI) merupakan standar komunikasi yang digunakan dalam pemrograman paralel pada sistem terdistribusi, yang memungkinkan beberapa proses untuk saling berkomunikasi dan bertukar data selama eksekusi berlangsung [10]. MPI menyediakan fungsi komunikasi seperti broadcast, scatter, dan gather yang sangat relevan dalam implementasi KNN paralel [14].

Lesmana et al. (2017) telah membuktikan bahwa pendekatan komputasi paralel mampu meningkatkan efisiensi pengolahan data prestasi akademik mahasiswa secara signifikan dibandingkan pendekatan sekuensial [17]. Namun, penelitian tersebut belum secara spesifik mengintegrasikan MPI dengan algoritma KNN untuk keperluan klasifikasi performa siswa. Dewi dan Sirait (2025) dalam penelitiannya tentang klasifikasi tingkat literasi siswa menggunakan KNN juga menyimpulkan bahwa algoritma ini memerlukan optimasi komputasi lebih lanjut ketika dihadapkan pada dataset berukuran besar [6]. Berdasarkan celah yang ditemukan

pada penelitian-penelitian tersebut, penelitian ini mengusulkan kombinasi KNN dan MPI sebagai solusi yang belum pernah diimplementasikan secara spesifik pada klasifikasi performa akademik siswa berskala besar, dengan kontribusi berupa bukti empiris bahwa pendekatan ini mampu mempertahankan akurasi tinggi hingga 96,82% sekaligus mereduksi waktu eksekusi secara signifikan dibandingkan pendekatan sekuensial.

III. METODOLOGI PENELITIAN

Penelitian ini menggunakan metodologi yang terdiri dari beberapa tahap, seperti yang ada pada *flowchart* berikut :



Gambar 1. Flowchart Metodologi

A. Pengumpulan dan Pemahaman Data

Dataset yang digunakan adalah data performa siswa yang terdiri dari 300.000 data. Dataset ini memiliki empat atribut fitur, yaitu `weekly_self_study_hours`, `attendance_percentage`, `class_participation`, dan `total_score`, serta `grade` sebagai label kelas. Klasifikasi dilakukan ke dalam lima kategori nilai: A, B, C, D, dan E. Pemilihan dataset ini didasarkan pada relevansinya terhadap permasalahan klasifikasi dalam bidang

pendidikan dan sifat numeriknya yang sesuai untuk diolah menggunakan KNN.

B. Pra-pemrosesan Data

Sebelum proses klasifikasi, data melalui tahap pra-pemrosesan yang meliputi penghapusan atribut yang tidak relevan (`student_id`) dan normalisasi data menggunakan metode Min-Max Scaling. Normalisasi penting dilakukan karena algoritma KNN sangat sensitif terhadap perbedaan skala data. Tanpa normalisasi, fitur dengan nilai besar dapat mendominasi perhitungan jarak dan menyebabkan hasil klasifikasi yang tidak akurat.

C. Pembagian Data dan Implementasi kNN

Dataset dibagi menjadi data latih dan data uji dengan rasio 80:20, di mana 80% data digunakan untuk melatih model dan 20% sisanya untuk pengujian. Algoritma KNN diimplementasikan dengan menentukan nilai K, menghitung Euclidean Distance antara data uji dan seluruh data latih, mengurutkan jarak, dan menentukan kelas berdasarkan mayoritas K tetangga terdekat.

D. Implementasi Komputasi Paralel dengan MPI

Untuk meningkatkan efisiensi komputasi, algoritma KNN diimplementasikan menggunakan pendekatan paralel dengan library `mpi4py`. Proses utama (rank 0) bertugas membaca data dan mendistribusikannya ke seluruh proses menggunakan mekanisme broadcast untuk data latih dan scatter untuk data uji. Setiap proses secara independen menghitung prediksi KNN untuk bagian data ujinya masing-masing. Hasil dari setiap proses dikumpulkan kembali ke proses utama menggunakan operasi reduce untuk dihitung akurasi dan waktu eksekusi akhirnya.

E. Evaluasi Model

Evaluasi dilakukan dengan mengukur waktu eksekusi program menggunakan fungsi waktu pada MPI untuk membandingkan performa antara pendekatan sekuensial dan paralel. Pengujian dilakukan dalam tiga skenario: (1) variasi jumlah core N dengan data tetap, (2) variasi jumlah data latih dengan

N=3 tetap, dan (3) variasi nilai K dengan N dan data tetap.

IV. HASIL DAN PEMBAHASAN

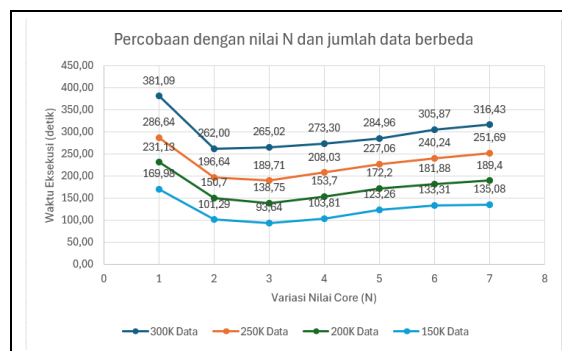
A. Pengaruh Jumlah Core (N) terhadap Waktu Eksekusi

Pengujian pertama dilakukan dengan memvariasikan jumlah core (N) dari 1 hingga 7 menggunakan data latih sebanyak 300.000 dan data uji 75.000 dengan nilai K=6. Hasil pengujian disajikan pada Tabel 1.

Tabel 1. Percobaan dengan nilai N dan jumlah data berbeda

N	150.000	200.000	250.000	300.000
1	169,98	231,13	286,64	381,09
2	101,29	150,7	196,64	262,00
3	93,64	138,75	189,71	265,02
4	103,81	153,7	208,03	273,30
5	123,26	172,2	227,06	284,96
6	133,31	181,88	240,24	305,87
7	135,08	189,4	251,69	316,43

Berdasarkan data pada Tabel di atas, dilakukan proses visualisasi data untuk mempermudah interpretasi hasil penelitian. Melalui visualisasi, perbedaan dan pola data dapat terlihat lebih jelas sehingga memudahkan analisis hasil.



Gambar 2. Percobaan dengan nilai N dan jumlah data berbeda

Berdasarkan **Gambar 2**, terdapat pola penurunan waktu eksekusi yang konsisten pada

seluruh ukuran dataset latih saat jumlah *core* ditingkatkan dari N=1 ke N=2 dan N=3. Pada dataset 300.000, waktu eksekusi berkurang drastis dari 381,09 detik pada N=1 dan mencapai titik optimal komputasinya pada N=2 dengan waktu 262,00 detik. Sementara itu, untuk dataset 250.000, 200.000, dan 150.000, performa optimal dicapai pada N=3 dengan waktu eksekusi terbaik masing-masing sebesar 189,71 detik, 138,75 detik, dan 93,64 detik. Pergeseran titik jenuh antara N=2 dan N=3 ini mengindikasikan bahwa keseimbangan antara beban komputasi dan *overhead* komunikasi MPI bersifat dinamis, sangat bergantung pada besaran ukuran dataset yang didistribusikan.

Seiring bertambahnya jumlah *core* melampaui titik optimal tersebut, waktu eksekusi justru meningkat secara bertahap pada semua skenario dataset. Pada dataset 300.000, waktu komputasi kembali naik dari 273,30 detik pada N=4 hingga mencapai 316,43 detik pada N=7. Pola peningkatan serupa juga terlihat jelas pada dataset 150.000, di mana waktu merangkak naik dari 103,81 detik pada N=4 menjadi 135,08 detik pada N=7. Peningkatan ini disebabkan oleh *overhead* komunikasi antar-proses yang membesar dan melampaui keuntungan waktu dari pembagian tugas, khususnya pada operasi *broadcast* dan *scatter* array data ke seluruh proses MPI, sehingga mencerminkan prinsip *diminishing returns* dalam eksekusi komputasi paralel.

B. Pengaruh Jumlah Data Latih terhadap Waktu Eksekusi

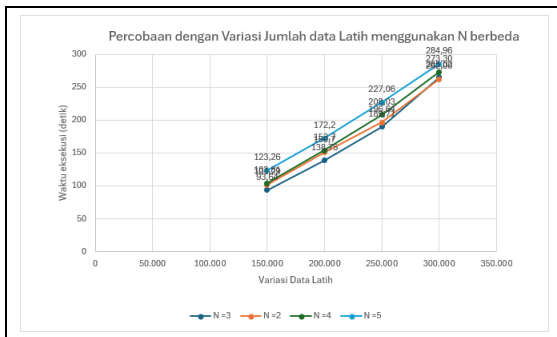
Pengujian kedua dilakukan dengan nilai N bervariasi (2, 3, 4, 5) dan memvariasikan jumlah data latih dari 100.000 hingga 300.000 dengan nilai K=5. Hasil pengujian disajikan pada Tabel 2.

Tabel 2. Hasil Pengujian Variasi Jumlah Data Latih Menggunakan N berbeda

Data Latih	N=2	N=3	N=4	N=5
150.000	101,29	93,64	103,81	123,26
200.000	150,7	138,75	153,7	172,2

250.000	196,64	189,71	208,03	227,06
300.000	262,00	265,02	273,30	284,96

Berdasarkan data pada Tabel di atas, dilakukan proses visualisasi data untuk mempermudah interpretasi hasil penelitian. Melalui visualisasi, perbedaan dan pola data dapat terlihat lebih jelas sehingga memudahkan analisis hasil.



Gambar 3. Percobaan dengan Variasi Jumlah Data Latih Menggunakan N berbeda

Berdasarkan **Gambar 3**, terdapat hubungan berbanding lurus antara jumlah data latih dengan waktu eksekusi pada seluruh konfigurasi *core* yang diuji. Pada N=3, waktu eksekusi meningkat secara konsisten dari 93,64 detik pada 150.000 data latih hingga mencapai 265,02 detik pada 300.000 data latih. Pola yang sama terjadi pada N=2, N=4, dan N=5, di mana waktu eksekusi masing-masing meningkat dari 101,29 detik, 103,81 detik, dan 123,26 detik pada 150.000 data hingga mencapai 262,00 detik, 273,30 detik, dan 284,96 detik pada 300.000 data. Hal ini terjadi karena setiap proses MPI tetap harus memproses bagian data yang semakin besar seiring bertambahnya jumlah data latih, sehingga beban komputasi algoritma KNN meningkat secara proporsional.

Pada seluruh ukuran dataset, N=3 secara konsisten menghasilkan waktu eksekusi terbaik dibandingkan konfigurasi lainnya, sementara N=2 berada di posisi kedua. Konfigurasi N=4 dan N=5 justru lebih lambat dibandingkan N=3 meskipun menggunakan lebih banyak core, yang kembali mengonfirmasi adanya overhead komunikasi antar-proses yang mendominasi pada jumlah core berlebih. Untuk menangani

data yang lebih masif, diperlukan strategi tambahan seperti pembagian array data uji yang dikirim, atau optimalisasi efisiensi algoritma untuk meminimalisir lonjakan *overhead* saat proses *broadcast* dan *scatter*.

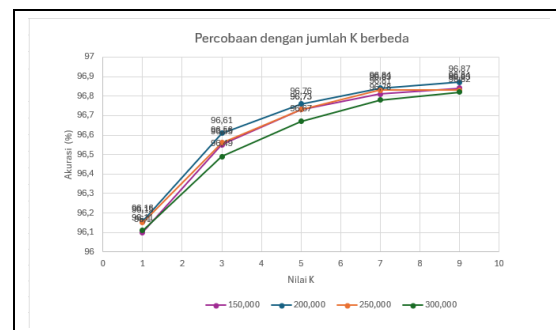
C. Pengaruh Nilai K terhadap Akurasi dan Waktu Eksekusi

Pengujian ketiga dilakukan dengan N=6 core dan data latih 300.000 yang tetap, kemudian memvariasikan nilai K dari 1 hingga 9. Hasil pengujian disajikan pada Tabel 3.

Tabel 3. Hasil Pengujian Variasi Nilai K (N=3, Data=300.000)

Jumlah Data Latih	K=1	K=3	K=5	K=7	K=9
150.000	96,10	96,5	96,7	96,8	96,8
0	%	5%	3%	1%	4%
200.000	96,1	96,6	96,7	96,8	96,8
0	6%	1%	6%	4%	7%
250.000	96,1	96,5	96,7	96,8	96,8
0	5%	6%	3%	3%	3%
300.000	96,1	96,4	96,6	96,7	96,8
0	1%	9%	7%	8%	2%

Berdasarkan data pada Tabel di atas, dilakukan proses visualisasi data untuk mempermudah interpretasi hasil penelitian. Melalui visualisasi, perbedaan dan pola data dapat terlihat lebih jelas sehingga memudahkan analisis hasil.



Gambar 4. Percobaan dengan Variasi Nilai K pada jumlah Data Latih Berbeda

Berdasarkan Gambar 3, perubahan nilai K berdampak pada akurasi klasifikasi model. Pada penggunaan 300.000 data latih, akurasi terendah tercatat pada K=1 sebesar 96,11%, kemudian meningkat secara konsisten pada K=3 menjadi 96,49%, K=5 sebesar 96,67%, K=7 sebesar 96,78%, hingga mencapai 96,82% pada K=9. Pola kenaikan yang stabil tanpa fluktuasi ini membuktikan bahwa penambahan jumlah tetangga referensi mampu menghasilkan prediksi yang lebih presisi, karena keputusan klasifikasi didasarkan pada konsensus titik data yang lebih banyak.

V. KESIMPULAN DAN SARAN

Berdasarkan hasil eksperimen dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan. Pertama, implementasi komputasi paralel dengan MPI pada algoritma KNN terbukti efektif meningkatkan efisiensi waktu eksekusi dalam mengklasifikasikan performa akademik siswa. Penambahan jumlah *core* mampu menurunkan waktu eksekusi secara signifikan, dengan waktu terbaik dicapai pada N=2 sebesar 262,00 detik dibandingkan N=1 yang membutuhkan 381,09 detik. Namun, penambahan *core* melampaui titik optimal justru meningkatkan waktu eksekusi akibat *overhead* komunikasi antar proses. Kedua, paralelisasi dengan MPI berhasil menjaga stabilitas akurasi klasifikasi, dengan akurasi tertinggi sebesar 96,87% pada K=9, menunjukkan bahwa distribusi beban kerja ke beberapa *core* tidak mengganggu kinerja prediksi. Ketiga, peningkatan volume data latih berbanding lurus dengan waktu eksekusi, di mana waktu meningkat dari 93,64 detik pada 150.000 data hingga 265,02 detik pada 300.000 data, karena beban komputasi pada setiap *core* bertambah seiring besarnya data yang diproses. Keempat, variasi nilai K menunjukkan tren akurasi yang meningkat secara konsisten dari 96,11% pada K=1 hingga 96,82% pada K=9, dengan selisih yang relatif kecil sebesar 0,71%, mengindikasikan bahwa model tidak terlalu sensitif terhadap perubahan nilai K.

Untuk penelitian selanjutnya, disarankan agar dilakukan pengujian pada dataset yang lebih besar dengan menggunakan infrastruktur komputasi terdistribusi multi-node untuk

mengoptimalkan potensi MPI. Selain itu, perlu dipertimbangkan strategi pembagian partisi data yang lebih seimbang guna meminimalkan overhead komunikasi antar proses dan meningkatkan efisiensi sistem secara keseluruhan.

VI. UCAPAN TERIMA KASIH (OPTIONAL)

Penulis mengucapkan terima kasih kepada Bapak/Ibu Dosen Program Studi Teknik Informatika, Fakultas Teknik, Universitas Mataram yang telah memberikan bimbingan, arahan, dan masukan selama proses penelitian dan penulisan makalah ini berlangsung. Penulis juga menyampaikan terima kasih kepada rekan-rekan mahasiswa yang telah memberikan dukungan dan diskusi yang konstruktif. Tidak lupa, penulis mengucapkan terima kasih kepada pihak penyelenggara SENIFORMA 2026, Seminar Nasional Informatika Universitas Mataram, yang telah menyediakan wadah bagi penulis untuk mempublikasikan hasil penelitian ini.

DAFTAR PUSTAKA

- [1] F. Tatimu, V. R. Palilingan, dan I. Rianto, "Analisis Klasifikasi Kelulusan Mahasiswa menggunakan Algoritma K-Nearest Neighbor di Fakultas Teknik Universitas Negeri Manado," *Journal of Education Method and Technology*, vol. 3, 2020.
- [2] D. Setiyadi dan A. Nurdin, "Data Mining Potensi Akademik Siswa Berbasis Online," vol. 2, no. 1, 2012.
- [3] M. R. Arrohman, N. Khoiriyah, A. Fawaida, dan D. L. Fithri, "Analisis Klasifikasi Kelulusan Siswa Menggunakan Metode KNN (K-Nearest Neighbor) Random Forest pada Dataset Students Performance," *Seminar Nasional Teknologi Informasi*, 2023.
- [4] T. Hidayat, Y. Handayani, dan A. Syaifudin, "Implementasi Algoritma K-Nearest Neighbor," *Jurnal Ilmiah Rekayasa dan Manajemen Sistem Informasi*, vol. 9, no. 2, hlm. 118–124, 2023.
- [5] A. N. Hilmi, E. Y. Puspaningrum, dan H. E. Wahanani, "Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Identifikasi Penyakit pada Tanaman Jeruk Berdasarkan Citra Daun," *Router: Jurnal Teknik Informatika dan Terapan*, vol. 2, no. 2, hlm.

- 107–117, 2024.
- [6] R. Dewi dan A. Sirait, "Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Klasifikasi Tingkat Literasi Siswa," *Jurnal Teknik Informatika dan Teknologi Informasi*, vol. 5, no. 3, hlm. 56–70, 2025.
- [7] A. Kurniati dan A. Hakim, "Analisis kemiskinan multidimensi di Kota Magelang dengan metode Multidimensional Poverty Index (MPI)," *Jurnal Kebijakan Ekonomi dan Keuangan*, hlm. 237–246, 2026.
- [8] S. Suliman, "Implementasi Data Mining Terhadap Prestasi Belajar Mahasiswa Berdasarkan Pergaulan dan Sosial Ekonomi Dengan Algoritma K-Means Clustering," *SIMKOM*, vol. 6, no. 1, hlm. 1–11, 2021.
- [9] D. Puspita Sari, S. Shofia Hilabi, dan A. Hananto, "Penerapan Data Mining Metode K-Nearest Neighbor Untuk Memprediksi Kelulusan Siswa Sekolah Menengah Pertama," *SMARTICS Journal*, vol. 9, no. 1, hlm. 14–19, 2023.
- [10] M. A. Wardana dan S. Suherman, "Penerapan Data Mining Pengelompokan Data Penjualan Motor di PT. Nusantara Surya Sakti Soppeng Menggunakan Metode K-Means Clustering," *JISTI*, vol. 8, no. 1, hlm. 123–132, 2025.
- [11] A. De Wibowo Muhammad Sidik, I. Himawan Kusumah, A. Suryana, M. Artiyasa, dan A. P. Junfithrana, "Gambaran Umum Metode Klasifikasi Data Mining," vol. 2, no. 2, hlm. 34–38, 2020.
- [12] J. Homepage dan B. Prasetyo, "Implementation of K-Nearest Neighbor Classification Algorithm as Decision Support Method in Staple Food Aid Distribution to Society," *IJRSE: Indonesian Journal of Informatic Research and Software Engineering*, 2022.
- [13] A. Khairi, I. Fahrezi, I. Sahputra, dan F. Anshari, "Implementation of K-Nearest Neighbor (KNN) Method to Determine House Locations in Batuphat and Tambon Tunong Areas, Aceh," *Jurnal Bisnis Informatika*, vol. 10, no. 1, 2021.
- [14] R. S. Daulay, "Analisis Kritis dan Pengembangan Algoritma K-Nearest Neighbor (KNN): Sebuah Tinjauan Literatur," *Jurnal Pendidikan Sains dan Komputer*, vol. 4, no. 02, hlm. 131–141, 2024.
- [15] P. I. Sijabat, E. Br Barus, Y. Laora Slingga, dan D. A. Putri, "Penerapan Algoritma K-Nearest Neighbor (KNN) untuk Menganalisis Pendapat Siswa terhadap Edukasi Anti Perundungan," *Jurnal Sistem Informasi TGD*, vol. 5, no. 1, hlm. 217–222, 2026.
- [16] H. Said, N. Matondang, H. Nurramdhani Irmanda, dan S. Informasi, "Penerapan Algoritma K-Nearest Neighbor untuk Memprediksi Kualitas Air yang Dapat Dikonsumsi," vol. 21, no. 2, 2022.
- [17] A. Lesmana dkk., "Komputasi Paralel untuk Pengolahan Prestasi Akademik Mahasiswa," 2017.
- [18] I. N. Sofiyanto, A. B. Saputra, R. F. Rakhmadianto, H. Nur, E. Febrianto, A. M. Endiwan, dan A. A. Sari, "Prediksi Kelulusan Mahasiswa Menggunakan Algoritma K-Nearest Neighbor (KNN) dengan Klasifikasi Biner," dalam *Prosiding Seminar Nasional Teknologi Informasi dan Bisnis (SENATIB)*, 2025.